

FIAP · Pós-Tech · IA para Devs

Detecção de Síndrome dos Ovários Policísticos (SOP/PCOS) com Machine Learning

Tech Challenge — Fase 1 · Módulo de Machine Learning

Grupo 21

Marcos de Lima · Michel Douglas Grigoli · Fabricio Geraldo Araujo

Agosto de 2026

1. Entregáveis e links

Repositório Git	https://github.com/michel005/fiap-tech-chalange-1-ia-para-devs
Vídeo de apresentação	https://www.youtube.com/watch?v=Du_KeZ0IGjA&list=PLdZIFpQTvILM (até 15 min)
Notebook principal	<code>notebooks/pcos_tech_challenge.ipynb</code>
Modelo treinado	<code>notebooks/modelo_pcos_final.pkl</code>
Dataset (no repositório)	<code>datasets/PCOS_data_without_infertility.xlsx</code> · aba <code>Full_new</code>
Dataset (download original)	Kaggle — Polycystic Ovary Syndrome (PCOS)
Enunciado do desafio	<code>docs/IADT-tech-challenge-fase-1.pdf</code>

Sobre o dataset

Base pública do Kaggle com **541 pacientes** e **45 colunas** originais. Cada linha é uma paciente; as colunas reúnem dados físicos (idade, peso, altura, IMC), exames de sangue e hormonais (beta-HCG, AMH, FSH, LH, TSH, PRL, vitamina D, etc.), dados de ultrassom (número de folículos no ovário esquerdo e direito, tamanho médio dos folículos, endométrio) e sintomas/hábitos (ganho de peso, queda de cabelo, crescimento de pelos, escurecimento da pele, acne, fast food, exercício). A variável-alvo é `PCOS (Y/N)` — 1 = tem SOP, 0 = não tem.

2. Instruções de execução (README)

O projeto roda com Python 3.10+ e Jupyter. Passo a passo a partir de um clone limpo do repositório:

```
# 1. Clonar o repositório
git clone https://github.com/michel005/fiap-tech-challenge-1-ia-para-devs.git
cd fiap-tech-challenge-1-ia-para-devs

# 2. Criar e ativar um ambiente virtual
python -m venv venv
# Windows (PowerShell):
venv\Scripts\Activate.ps1
# Linux / macOS:
source venv/bin/activate

# 3. Instalar as dependências
pip install -r requirements.txt
# O notebook também usa SHAP e a leitura de Excel (.xlsx):
pip install shap openpyxl

# 4. Abrir o Jupyter e executar o notebook principal
jupyter notebook notebooks/pcos_tech_challenge.ipynb
# No menu: Kernel > Restart & Run All
```

Observações

- O arquivo `datasets/PCOS_data_without_infertility.xlsx` já está versionado no repositório; o notebook o lê pelo caminho relativo `../datasets/PCOS_data_without_infertility.xlsx`, portanto basta rodar as células na ordem.
- Dependências do notebook: `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`, `shap`, `openpyxl`, `jupyter`.
- A semente aleatória é fixada em `RANDOM_STATE = 42`, então os resultados são reproduzíveis a cada execução.

3. Relatório técnico

3.1 Definição do problema

O desafio simula uma rede de hospitais e centros de saúde da mulher que precisa de um sistema inteligente para identificar precocemente condições que afetam a saúde feminina. Escolhemos a **Síndrome dos Ovários Policísticos (SOP / PCOS)**, um dos distúrbios hormonais mais comuns em mulheres em idade reprodutiva (estima-se de 6% a 12% no mundo). Quando não detectada cedo, está associada a infertilidade, diabetes tipo 2 e doenças cardiovasculares.

O objetivo é construir um modelo de classificação que, a partir de exames e características clínicas, estime a **probabilidade de a paciente ter SOP**, funcionando como **ferramenta de apoio à triagem** — para priorizar quem precisa de investigação mais rápida. O modelo **não substitui o médico**: a decisão final é sempre clínica.

3.2 Discussões da análise exploratória (EDA)

- **Estrutura e tipos.** `df.info()` revelou que colunas que deveriam ser numéricas (`II beta-HCG(mIU/mL)` e `AMH(ng/mL)`) estavam como `object` (texto), por causa de valores digitados de forma inválida no arquivo original (por exemplo `"1.99."` com ponto a mais, ou uma letra `"a"`). Havia ainda a coluna `Unnamed: 44` , praticamente vazia (apenas 2 valores não-nulos em 541 linhas), e duas colunas de identificação (`Sl. No` e `Patient File No.`) sem valor preditivo.
- **Distribuição do alvo.** 364 pacientes sem SOP (67,3%) e 177 com SOP (32,7%) — um **desbalanceamento moderado**. Isso é decisivo para a avaliação: um classificador que sempre respondesse "não tem SOP" atingiria 67% de acurácia sem nenhuma utilidade clínica. Daí a decisão de estratificar a divisão treino/teste e priorizar Recall e F1 em vez de acurácia (ver Figura 1).
- **Correlação com o alvo.** As variáveis mais correlacionadas com `PCOS (Y/N)` foram `Follicle No. (R)` (0,648) e `Follicle No. (L)` (0,603), seguidas de escurecimento da pele (0,476), crescimento de pelos / hirsutismo (0,465), ganho de peso (0,441) e irregularidade do ciclo menstrual (0,402). Esse ranking coincide com os critérios diagnósticos reais da SOP (critérios de Rotterdam: hiperandrogenismo, disfunção ovulatória e morfologia ovariana policística). O fato de os dados "fazerem sentido clínico" aumenta a confiança para seguir com a modelagem (ver Figura 2).

3.3 Estratégias de pré-processamento

1. **Normalização dos nomes das colunas** com `str.strip()` , removendo espaços extras que vinham do arquivo Excel e causavam erros ao referenciar colunas.
2. **Remoção de colunas sem valor preditivo:** `Sl. No` , `Patient File No.` (identificadores) e `Unnamed: 44` (coluna vazia). Passamos de 45 para 42 colunas.
3. **Conversão de tipos** com `pd.to_numeric(..., errors="coerce")` nas colunas de exame lidas como texto (`II beta-HCG` e `AMH`). Os valores inválidos viram `NaN` automaticamente, em vez de quebrar o pipeline.
4. **Imputação de valores ausentes pela mediana** de cada coluna (`fillna(df.median())`). Optou-se pela mediana, e não pela média, por ser mais robusta a *outliers* — comuns em exames hormonais (a

beta-HCG, por exemplo, pode ter picos muito altos). Resultado: 541 pacientes, 42 colunas, **zero valores ausentes** e todas as colunas numéricas.

5. **Padronização de escala com `StandardScaler` dentro de um `Pipeline`** do scikit-learn. Colocar o scaler no pipeline garante que ele seja ajustado *apenas* nos dados de treino a cada fold/execução, evitando *data leakage* (vazamento de informação do teste para o treino).
6. **Análise de correlação** também como etapa de pré-processamento: ajuda a identificar variáveis relevantes e possível multicolinearidade (variáveis que medem quase a mesma coisa, como os pares de folículos e de pressão arterial).

3.4 Modelos usados e por quê

O enunciado exige pelo menos dois algoritmos de classificação; usamos **três**, de famílias diferentes, para uma comparação mais rica. Todos foram encapsulados em `Pipeline(StandardScaler + modelo)`.

Modelo	Família	Por que foi escolhido
Regressão Logística	Linear	Simples, rápido e altamente interpretável — os coeficientes indicam diretamente o peso de cada variável. Bom baseline e alinhado à exigência de explicabilidade em saúde.
Random Forest	Ensemble de árvores	Captura relações não-lineares e interações, é robusto a outliers e fornece <code>feature_importances_</code> nativamente. Referência de desempenho.
KNN (k=5)	Baseado em instâncias	Abordagem não-paramétrica: classifica a paciente pelos casos mais parecidos. Serve de contraponto e é sensível à escala (daí a importância do scaler).

Separação treino/teste: `train_test_split` com `test_size=0.2` (432 pacientes para treino, 109 para teste) e `stratify=y`, mantendo ~33% de casos positivos nos dois conjuntos (32,6% no treino, 33,0% no teste).

3.5 Escolha da métrica principal

Em diagnóstico médico os erros têm pesos diferentes:

- **Falso Negativo** — dizer que a paciente NÃO tem SOP quando ela TEM. É o erro mais grave: um diagnóstico perdido.
- **Falso Positivo** — dizer que tem SOP quando não tem. Menos grave: gera exames adicionais, mas não deixa a doença passar.

Por isso a métrica priorizada é o **Recall da classe positiva** (proporção de pacientes com SOP que o modelo identifica), com o **F1-score** como equilíbrio para não "chutar SOP para todo mundo". A acurácia isolada é enganosa sob desbalanceamento. O AUC é reportado como medida global de separação entre as classes.

3.6 Resultados e interpretação

Modelo	Accuracy	Precision	Recall	F1	AUC
Regressão Logística	0,890	0,816	0,861	0,838	0,951
Random Forest	0,899	0,903	0,778	0,836	0,948

KNN	0,899	0,963	0,722	0,825	0,937
-----	-------	-------	-------	-------	-------

Modelo final escolhido: Regressão Logística, por ter o maior Recall (0,861) e o melhor F1 (0,838), além do maior AUC (0,951). Random Forest e KNN têm Precision mais alta, mas ao custo de mais Falsos Negativos — o oposto do que este problema exige.

No conjunto de teste (109 pacientes, 36 com SOP), a Regressão Logística obteve, para a classe PCOS: precision 0,82, recall 0,86, f1 0,84. As matrizes de confusão (Figura 4) mostram **5 Falsos Negativos** para a Regressão Logística, contra 8 do Random Forest e 10 do KNN. Interpretação: de cada 100 pacientes que realmente têm SOP, o modelo final identifica cerca de 86 — ao preço de alguns alarmes falsos, aceitáveis num contexto de triagem.

3.7 Explicabilidade — Feature Importance e SHAP

Feature Importance (Figura 6): para a Regressão Logística usamos o valor absoluto dos coeficientes. As cinco variáveis mais influentes foram `Follicle No. (R)` (1,755), `hair growth(Y/N)` (0,785), `Follicle No. (L)` (0,686), `Cycle(R/I)` (0,678) e `Weight gain(Y/N)` (0,656) — exatamente os fatores associados à SOP na prática clínica. O modelo aprendeu padrões coerentes, não atalhos espúrios.

SHAP (*SHapley Additive exPlanations*, Figura 7): vai além da importância global e mostra a contribuição de cada variável para cada paciente. No *summary plot*, cada ponto é uma paciente; pontos à direita empurram a previsão para "tem SOP" e a cor indica o valor da variável (vermelho = alto). Valores altos de número de folículos e dos marcadores de hirsutismo/ganho de peso aparecem à direita, confirmando visual e individualmente o que a feature importance mostrou de forma agregada. Isso torna o modelo **auditável**: o profissional pode comparar a explicação com o próprio julgamento clínico.

3.8 Discussão crítica e conclusões

O modelo pode ser usado na prática? Sim, como **ferramenta de apoio à triagem**, não como decisor final. Ao dar entrada no sistema, os dados da paciente passariam pelo modelo e as de alta probabilidade seriam encaminhadas com prioridade ao especialista, acelerando o atendimento em locais de grande volume.

Limite ético. O médico tem sempre a palavra final. Nenhum modelo de ML substitui exames, avaliação clínica e julgamento profissional.

Limitações. (1) Dataset pequeno (~541 pacientes); (2) origem única — não se sabe se os padrões generalizam para outras populações, regiões ou países, sendo necessária validação externa antes de qualquer implantação; (3) possíveis vieses da fonte de dados que desconhecemos.

Próximos passos. Coletar mais dados de diferentes hospitais; ajuste de hiperparâmetros; ajuste do limiar de decisão para maximizar ainda mais o Recall; e monitoramento contínuo caso o modelo entre em uso.

Resumo do pipeline entregue: carga e exploração dos dados → pré-processamento (limpeza, conversão de tipos, imputação, escala) → treinamento de três classificadores → avaliação com métricas adequadas ao contexto médico → explicabilidade com Feature Importance e SHAP → discussão crítica.

4. Resultados obtidos — prints, gráficos e análises

Saídas numéricas principais (conjunto de teste, `random_state=42`):

Modelo	Accuracy	Precision	Recall	F1	AUC
Regressão Logística	0.890	0.816	0.861	0.838	0.951
Random Forest	0.899	0.903	0.778	0.836	0.948
KNN	0.899	0.963	0.722	0.825	0.937

Modelo final: Regressão Logística					
	precision	recall	f1-score	support	
Não PCOS (0)	0.93	0.90	0.92	73	
PCOS (1)	0.82	0.86	0.84	36	
accuracy			0.89	109	

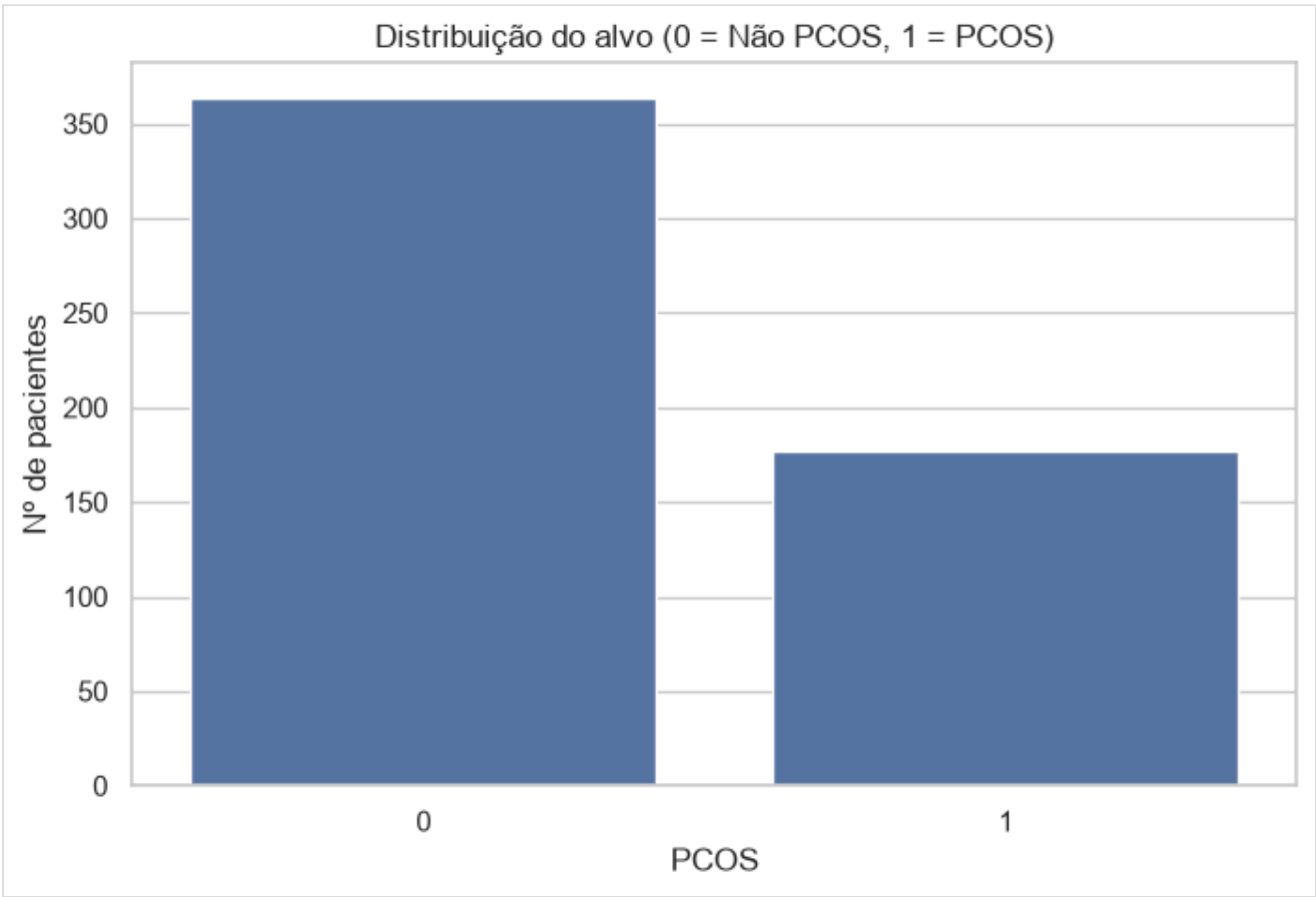


Figura 1 — Distribuição da variável-alvo PCOS (0 = não tem, 1 = tem). 364 pacientes sem SOP (67,3%) e 177 com SOP (32,7%): desbalanceamento moderado.

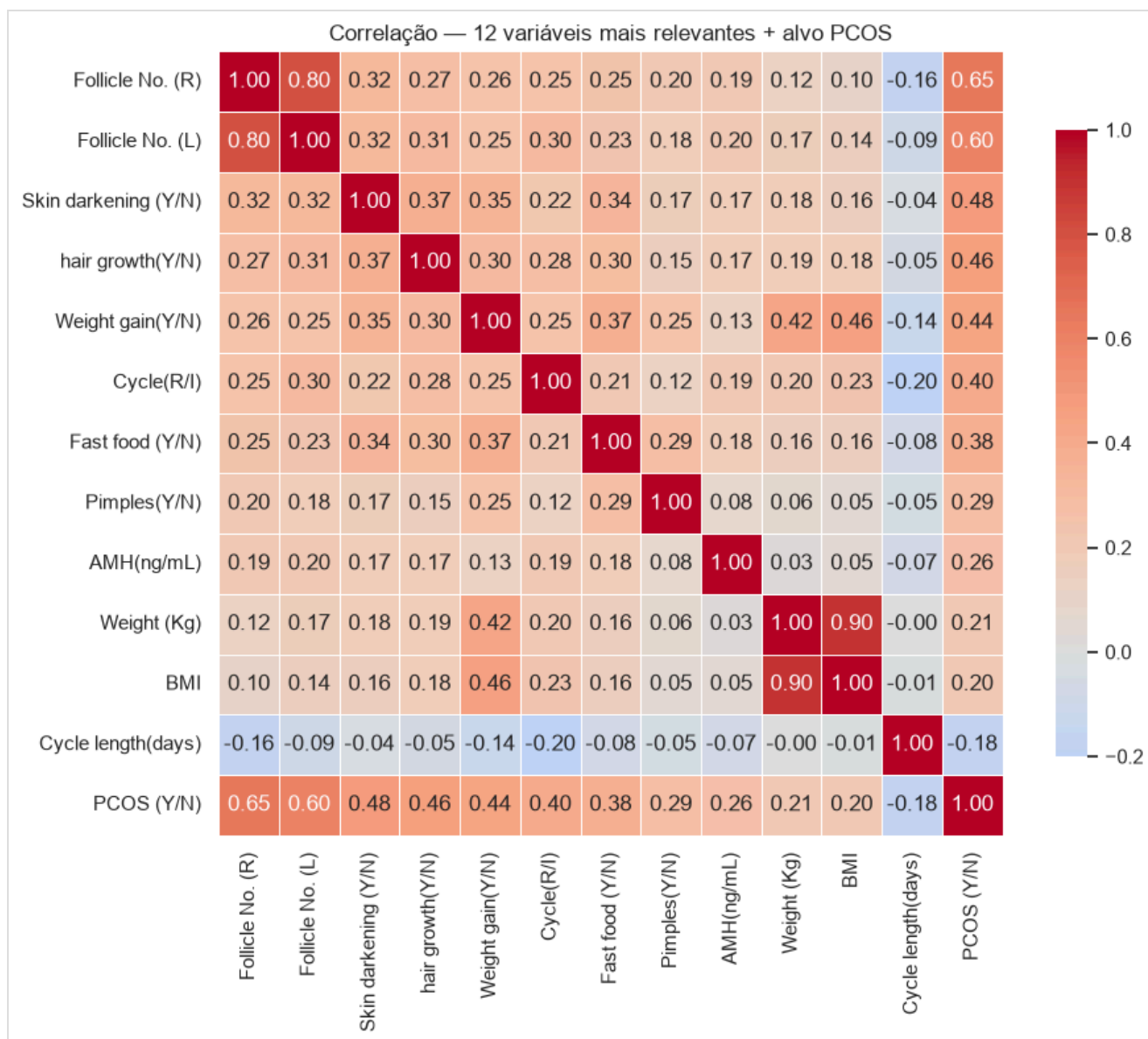


Figura 2 — Heatmap de correlação das 12 variáveis mais relevantes com o alvo. N° de folículos (R/L), ganho de peso, irregularidade do ciclo e escurecimento da pele lideram — coerente com o quadro clínico da SOP.

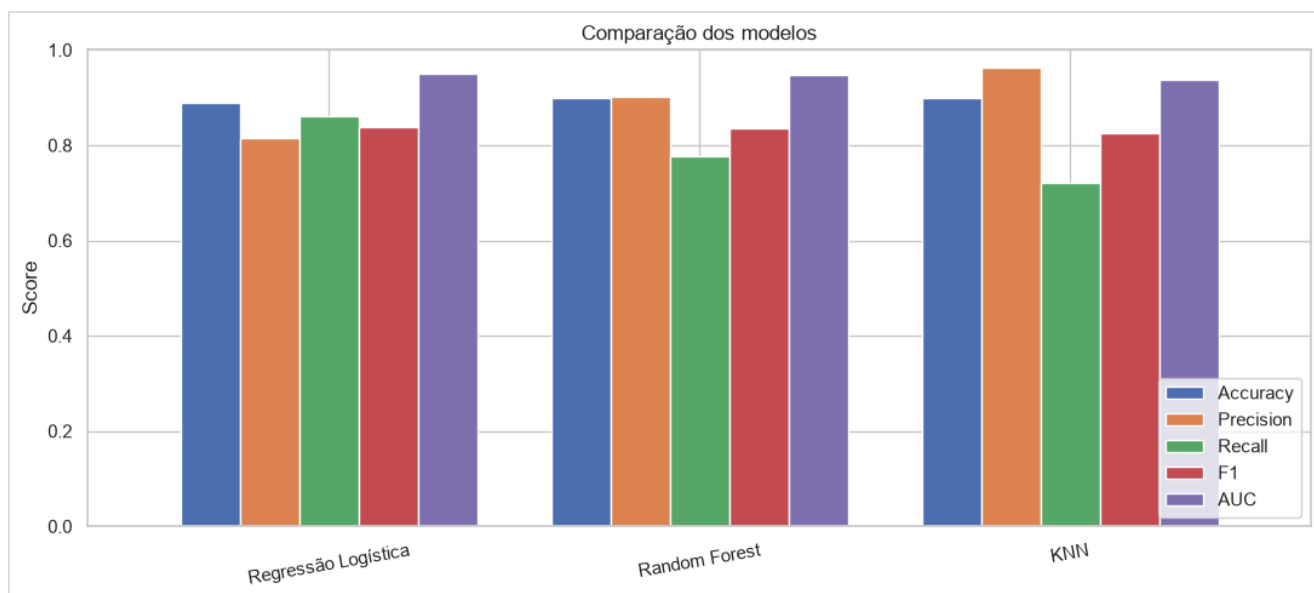


Figura 3 — Comparação das métricas dos três modelos. A Regressão Logística tem o maior Recall (0,861); Random Forest e KNN têm Precision maior, mas Recall menor.

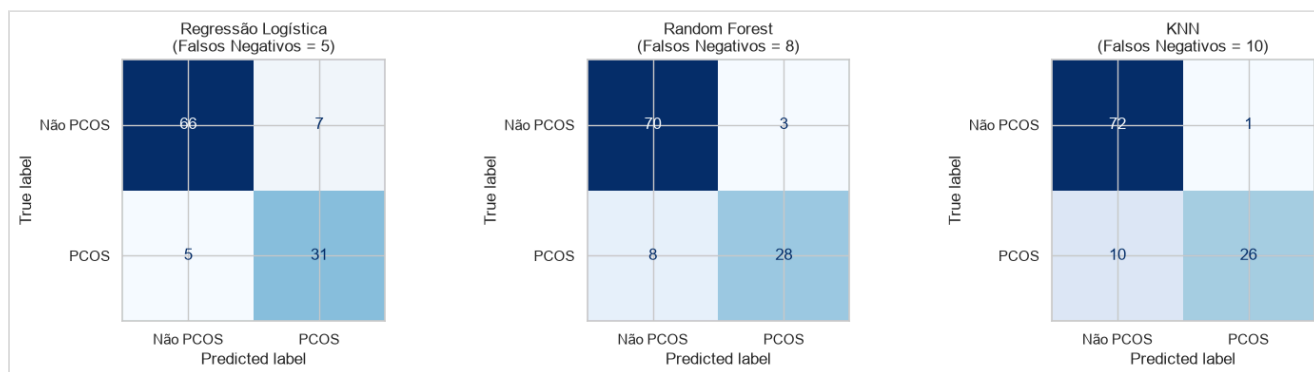


Figura 4 — Matrizes de confusão. O foco é o canto inferior esquerdo (Falsos Negativos): Regressão Logística = 5, Random Forest = 8, KNN = 10.

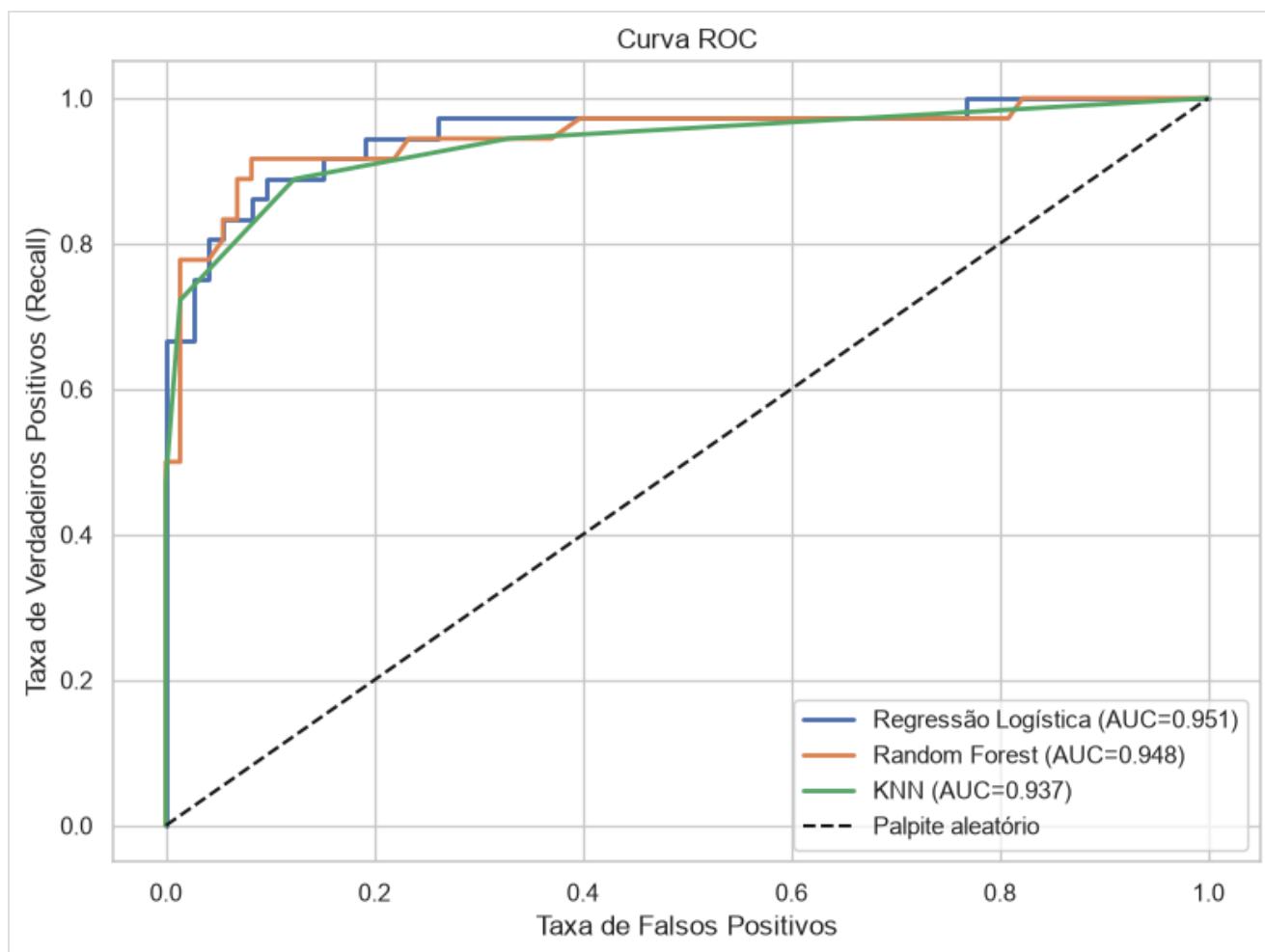


Figura 5 — Curvas ROC. Regressão Logística AUC = 0,951; Random Forest = 0,948; KNN = 0,937. Todas bem acima do palpite aleatório.

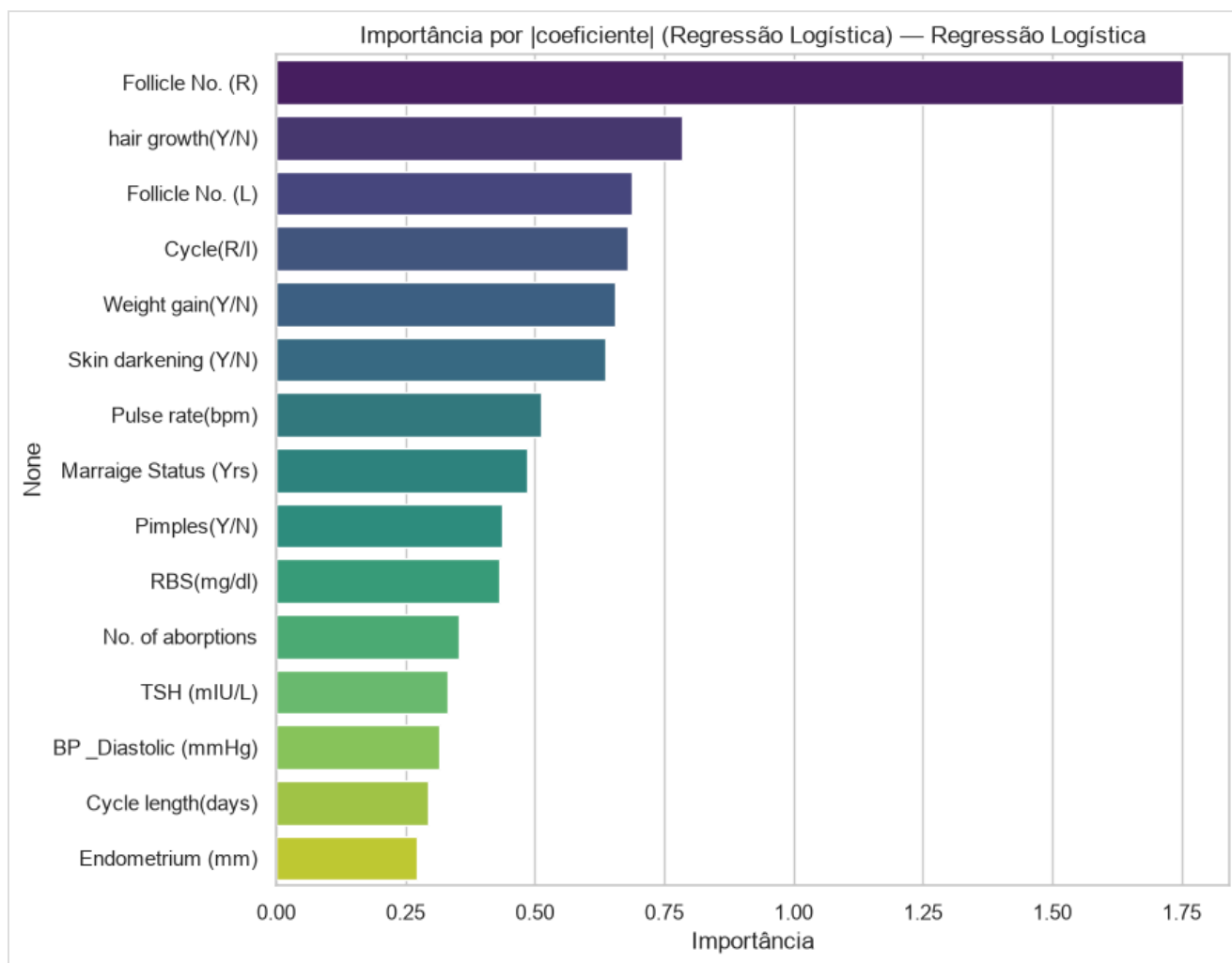


Figura 6 — Importância das variáveis (|coeficiente| da Regressão Logística). Top 5: Follicle No. (R) 1,755; hair growth 0,785; Follicle No. (L) 0,686; Cycle(R/I) 0,678; Weight gain 0,656.

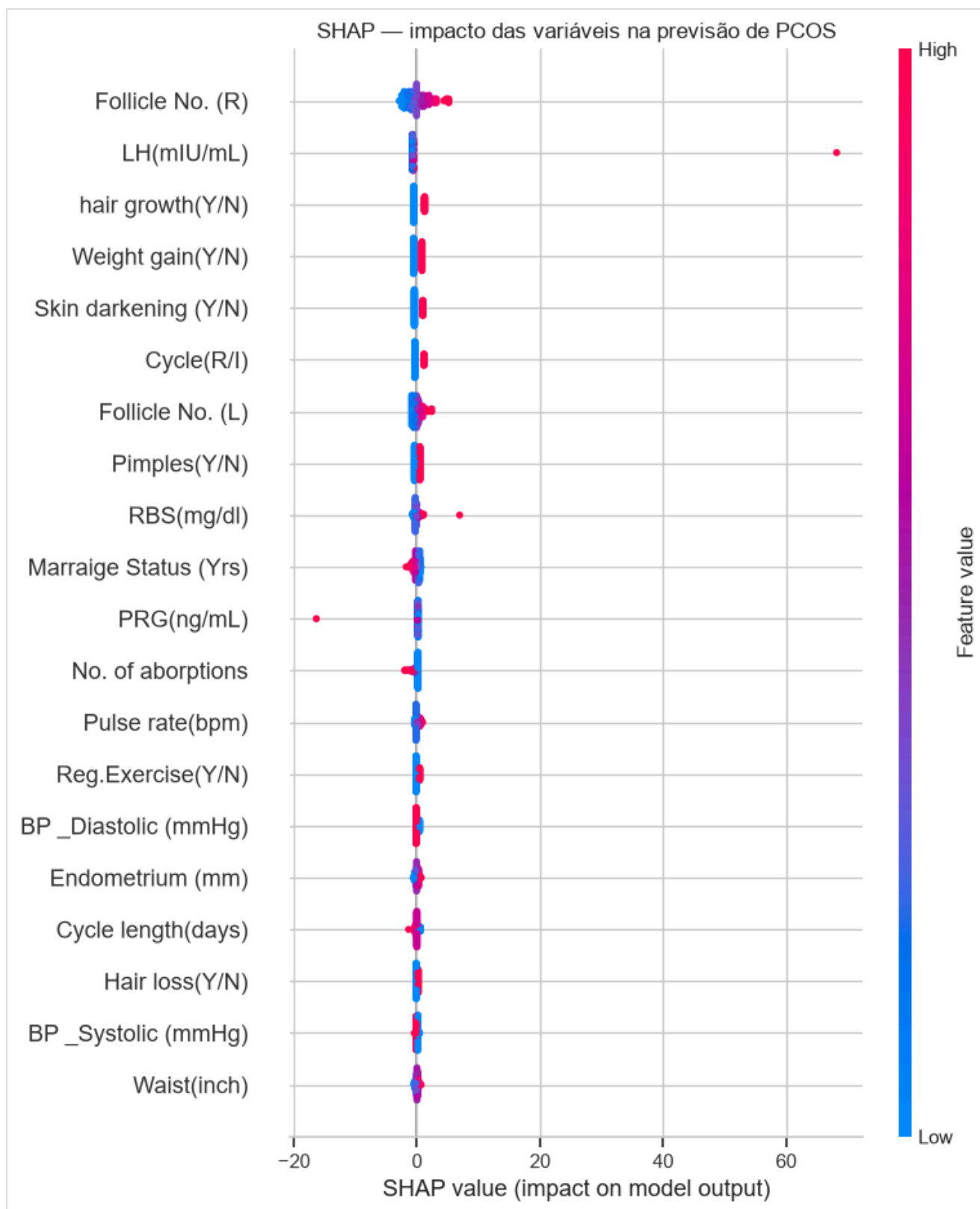


Figura 7 — SHAP summary plot (classe PCOS = 1). Cada ponto é uma paciente; à direita empurra para "tem SOP". Vermelho = valor alto da variável. Muitos folículos e sinais de hirsutismo/ganho de peso elevam a probabilidade prevista.

5. Código-fonte completo

Transcrição integral do notebook `notebooks/pcos_tech_challenge.ipynb` (células de texto, código e saídas).

Detecção de Síndrome dos Ovários Policísticos (SOP/PCOS) com Machine Learning

Tech Challenge — Fase 1 (Módulo de Machine Learning)

Este notebook constrói um modelo de Machine Learning capaz de apoiar a detecção da Síndrome dos Ovários Policísticos (SOP) — em inglês, *PCOS* — a partir de dados clínicos de pacientes.

O problema

A SOP é um dos distúrbios hormonais mais comuns entre mulheres em idade reprodutiva.

Quando não diagnosticada, pode levar a complicações como infertilidade, diabetes e doenças cardíacas. O diagnóstico, porém, depende de vários exames e da análise de um especialista, o que torna a triagem precoce um desafio.

A proposta aqui é criar uma ferramenta de apoio à decisão médica: um modelo que, a partir de exames e características da paciente, estima a probabilidade de SOP e ajuda a priorizar quem precisa de investigação mais rápida. O modelo **não substitui o médico** — ele apenas apoia a triagem.

O dataset

- **Fonte:** [PCOS — Kaggle](#)
- **Arquivo:** `PCOS_data_without_infertility.xlsx`, aba `Full_new`
- Contém dados de ~541 pacientes com exames hormonais, medidas físicas e resultados de

ultrassom. A coluna-alvo é `PCOS (Y/N)` (1 = tem SOP, 0 = não tem).

O que fizemos (passo a passo)

- Carga dos dados
- Exploração (estatísticas, distribuição do alvo, correlações)
- Pré-processamento (limpeza, conversão de tipos, valores ausentes)
- Modelagem (3 modelos de classificação + separação treino/teste)
- Avaliação (accuracy, recall, F1 e escolha da métrica)
- Explicabilidade (feature importance + SHAP)

- Discussão crítica

0. Preparação do ambiente

Aqui apenas importamos as bibliotecas que utilizei durante todo o projeto.

Se estiver rodando no Google Colab e faltar algum pacote, descomente a linha de `pip install`.

```
# Se algum pacote faltar (ex.: no Colab), descomente a linha abaixo:
# !pip install -q scikit-learn pandas numpy matplotlib seaborn shap openpyxl

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, recall_score, f1_score, precision_score,
                             confusion_matrix, ConfusionMatrixDisplay,
                             classification_report, roc_curve, roc_auc_score)

# Deixa os gráficos mais bonitos e define um "random_state" para reprodutibilidade
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
RANDOM_STATE = 42 # fixa a aleatoriedade -> resultados sempre iguais a cada execução

print("Bibliotecas importadas com sucesso!")
```

Bibliotecas importadas com sucesso!

1. Carga dos dados

O dataset vem em um arquivo Excel com várias abas. A aba com os dados completos é a

`Full_new`, então foi ela que carreguei.

```
# Caminho do arquivo (deve estar na mesma pasta do notebook)
ARQUIVO = "../datasets/PCOS_data_without_infertility.xlsx"

# Lemos a aba "Full_new", que contém os dados completos
df = pd.read_excel(ARQUIVO, sheet_name="Full_new")

print(f"Dados carregados: {df.shape[0]} pacientes e {df.shape[1]} colunas.")
df.head()
```

Dados carregados: 541 pacientes e 45 colunas.

	Sl. No	Patient File No.	PCOS (Y/N)	Age (yrs)	Weight (Kg)	Height(Cm)	\
0	1	1	0	28	44.6	152.0	
1	2	2	0	36	65.0	161.5	
2	3	3	1	33	68.8	165.0	
3	4	4	0	37	65.0	148.0	
4	5	5	0	25	52.0	161.0	

	BMI	Blood Group	Pulse rate(bpm)	RR (breaths/min)	...	\
0	19.300000	15	78	22	...	
1	24.921163	15	74	20	...	
2	25.270891	11	72	18	...	
3	29.674945	13	72	20	...	
4	20.060954	11	72	18	...	

	Fast food (Y/N)	Reg.Exercise(Y/N)	BP _Systolic (mmHg)	\
0	1.0	0	110	
1	0.0	0	120	
2	1.0	0	120	
3	0.0	0	120	
4	0.0	0	120	

	BP _Diastolic (mmHg)	Follicle No. (L)	Follicle No. (R)	\
0	80	3	3	
1	70	3	5	
2	80	13	15	
3	70	2	2	
4	80	3	4	

	Avg. F size (L) (mm)	Avg. F size (R) (mm)	Endometrium (mm)	Unnamed: 44
0	18.0	18.0	8.5	NaN
1	15.0	14.0	3.7	NaN
2	18.0	20.0	10.0	NaN
3	15.0	14.0	7.5	NaN
4	16.0	14.0	7.0	NaN

[5 rows x 45 columns]

2. Exploração dos dados (EDA)

Antes de modelar, precisei entender os dados: quantas colunas tinha, quais os tipos, se existem valores ausentes, como o alvo está distribuído e quais variáveis mais se relacionam com a SOP.

```
# Visão geral: tipos de dados e valores não-nulos de cada coluna
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541 entries, 0 to 540
Data columns (total 45 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Sl. No                                541 non-null    int64
1   Patient File No.                      541 non-null    int64
2   PCOS (Y/N)                            541 non-null    int64
3   Age (yrs)                             541 non-null    int64
4   Weight (Kg)                           541 non-null    float64
5   Height(Cm)                            541 non-null    float64
6   BMI                                    541 non-null    float64
7   Blood Group                           541 non-null    int64
8   Pulse rate(bpm)                       541 non-null    int64
9   RR (breaths/min)                      541 non-null    int64
10  Hb(g/dl)                              541 non-null    float64
11  Cycle(R/I)                            541 non-null    int64
12  Cycle length(days)                    541 non-null    int64
13  Marraige Status (Yrs)                  540 non-null    float64
14  Pregnant(Y/N)                          541 non-null    int64
15  No. of aborptions                      541 non-null    int64
16  I   beta-HCG(mIU/mL)                   541 non-null    float64
17  II  beta-HCG(mIU/mL)                   541 non-null    object
18  FSH(mIU/mL)                           541 non-null    float64
19  LH(mIU/mL)                            541 non-null    float64
20  FSH/LH                                541 non-null    float64
21  Hip(inch)                             541 non-null    int64
22  Waist(inch)                           541 non-null    int64
23  Waist:Hip Ratio                        541 non-null    float64
24  TSH (mIU/L)                           541 non-null    float64
25  AMH(ng/mL)                            541 non-null    object
26  PRL(ng/mL)                            541 non-null    float64
27  Vit D3 (ng/mL)                        541 non-null    float64
28  PRG(ng/mL)                            541 non-null    float64
29  RBS(mg/dl)                            541 non-null    float64
30  Weight gain(Y/N)                       541 non-null    int64
31  hair growth(Y/N)                       541 non-null    int64
32  Skin darkening (Y/N)                   541 non-null    int64
33  Hair loss(Y/N)                         541 non-null    int64
34  Pimples(Y/N)                           541 non-null    int64
35  Fast food (Y/N)                        540 non-null    float64
36  Reg.Exercise(Y/N)                      541 non-null    int64
37  BP _Systolic (mmHg)                    541 non-null    int64
38  BP _Diastolic (mmHg)                   541 non-null    int64
39  Follicle No. (L)                       541 non-null    int64
40  Follicle No. (R)                       541 non-null    int64
41  Avg. F size (L) (mm)                   541 non-null    float64
42  Avg. F size (R) (mm)                   541 non-null    float64
43  Endometrium (mm)                       541 non-null    float64
44  Unnamed: 44                            2 non-null     object
dtypes: float64(19), int64(23), object(3)
memory usage: 190.3+ KB
```



```
# Estatísticas descritivas das colunas numéricas (média, desvio, mínimo, máximo...)
# Mostra apenas as 10 primeiras para não poluir a tela.
df.describe().T.head(10).round(2)
```

	count	mean	std	min	25%	50%	75%	max
Sl. No	541.0	271.00	156.32	1.00	136.00	271.00	406.00	541.0
Patient File No.	541.0	271.00	156.32	1.00	136.00	271.00	406.00	541.0
PCOS (Y/N)	541.0	0.33	0.47	0.00	0.00	0.00	1.00	1.0
Age (yrs)	541.0	31.43	5.41	20.00	28.00	31.00	35.00	48.0
Weight (Kg)	541.0	59.64	11.03	31.00	52.00	59.00	65.00	108.0
Height(Cm)	541.0	156.48	6.03	137.00	152.00	156.00	160.00	180.0
BMI	541.0	24.31	4.06	12.42	21.64	24.24	26.63	38.9
Blood Group	541.0	13.80	1.84	11.00	13.00	14.00	15.00	18.0
Pulse rate(bpm)	541.0	73.25	4.43	13.00	72.00	72.00	74.00	82.0
RR (breaths/min)	541.0	19.24	1.69	16.00	18.00	18.00	20.00	28.0

2.1 Distribuição do alvo

Verificar como as classes estão distribuídas é essencial. Se houver desbalanceamento

(muito mais casos de uma classe do que da outra), a métrica accuracy pode enganar, e

precisa tomar cuidado na avaliação.

```
TARGET = "PCOS (Y/N)" # nome da coluna-alvo

# Contagem e proporção de cada classe
contagem = df[TARGET].value_counts()
proporcao = df[TARGET].value_counts(normalize=True).mul(100).round(1)

print("Contagem por classe (0 = Não tem SOP, 1 = Tem SOP):")
print(contagem, "\n")
print("Proporção (%):")
print(proporcao)

# Gráfico de barras da distribuição
sns.countplot(x=df[TARGET])
plt.title("Distribuição do alvo (0 = Não PCOS, 1 = PCOS)")
plt.xlabel("PCOS"); plt.ylabel("Nº de pacientes")
plt.show()
```

Contagem por classe (0 = Não tem SOP, 1 = Tem SOP):

PCOS (Y/N)

0 364

1 177

Name: count, dtype: int64

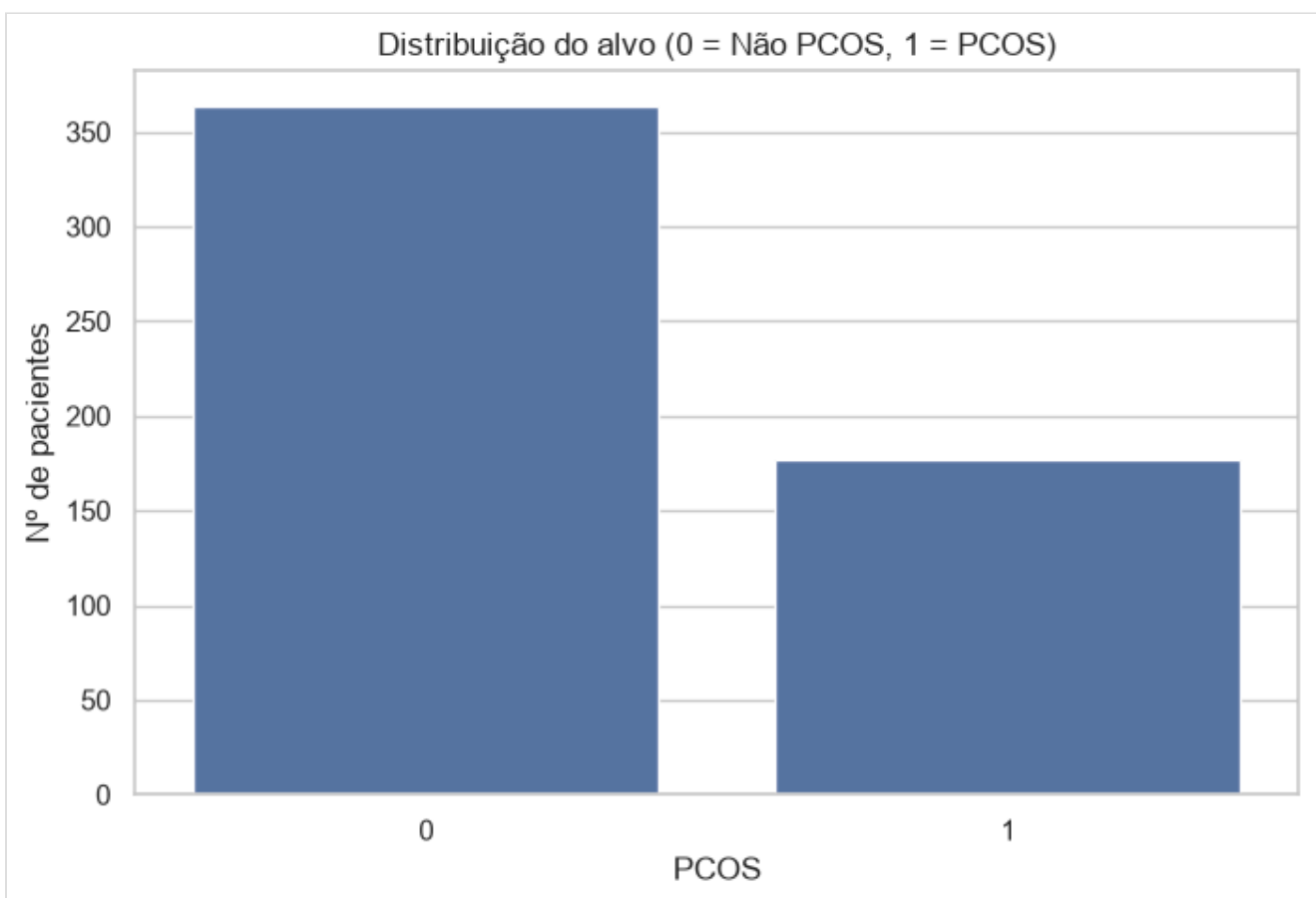
Proporção (%):

PCOS (Y/N)

0 67.3

1 32.7

Name: proportion, dtype: float64



Leitura: cerca de 1/3 das pacientes têm SOP (classe 1) e 2/3 não têm. É um

desbalanceamento moderado. Por isso, mais adiante, usei `stratify` na divisão treino/teste (para manter essa proporção) e dei atenção especial ao **Recall** e ao **F1**, e não só à accuracy.

2.2 Análise de correlação

A correlação mede o quanto duas variáveis "andam juntas". Aqui quis ver quais exames e características mais se relacionam com a presença de SOP — isso ajuda a validar se os dados fazem sentido do ponto de vista clínico.

Observação: a coluna alvo já é numérica e algumas colunas ainda estão como texto.

Fiz a análise de correlação de forma completa depois da limpeza (seção 3), mas já dei uma primeira olhada aqui.

```
# Correlação de cada coluna numérica com o alvo (em valor absoluto)
corr = df.corr(numeric_only=True)
corr_alvo = corr[TARGET].drop(TARGET).abs().sort_values(ascending=False)

print("Top 10 variáveis mais correlacionadas com PCOS:")
print(corr_alvo.head(10).round(3))
```

```
Top 10 variáveis mais correlacionadas com PCOS:
Follicle No. (R)      0.648
Follicle No. (L)      0.603
Skin darkening (Y/N)  0.476
hair growth(Y/N)     0.465
Weight gain(Y/N)     0.441
Cycle(R/I)           0.402
Fast food (Y/N)      0.378
Pimples(Y/N)         0.286
Weight (Kg)          0.212
BMI                  0.200
Name: PCOS (Y/N), dtype: float64
```

3. Pré-processamento (limpeza)

Dados reais quase nunca vêm prontos. Nesta etapa fiz o seguinte:

- Limpar os nomes das colunas (remover espaços extras);
- Remover colunas inúteis (números de identificação e uma coluna vazia);
- Converter para número as colunas de exame que vieram como texto por engano;
- Preencher valores ausentes usando a mediana de cada coluna.

Usei a mediana (e não a média) porque ela é mais robusta a valores extremos (*outliers*), comuns em exames hormonais.

```

linhas0, cols0 = df.shape

# 3.1 Remover espaços extras nos nomes das colunas
df.columns = df.columns.str.strip()

# 3.2 Remover colunas que não ajudam a prever:
#     - "Sl. No" e "Patient File No." são apenas identificadores
#     - "Unnamed: 44" é uma coluna vazia
cols_remove = ["Sl. No", "Patient File No.", "Unnamed: 44"]
df = df.drop(columns=[c for c in cols_remove if c in df.columns], errors="ignore")

# 3.3 Converter para número as colunas de exame que vieram como texto.
#     errors="coerce" transforma valores inválidos (ex.: digitação errada) em NaN.
cols_corrigir = ["II      beta-HCG(mIU/mL)", "AMH(ng/mL)"]
for col in cols_corrigir:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# 3.4 Preencher valores ausentes com a MEDIANA de cada coluna
df = df.fillna(df.median(numeric_only=True))

print(f"Antes: {linhas0} linhas x {cols0} colunas")
print(f"Depois: {df.shape[0]} linhas x {df.shape[1]} colunas")
print(f"Valores ausentes restantes: {int(df.isna().sum().sum())}")
print(f"Colunas de texto restantes: {len(df.select_dtypes(include='object').columns)}")

```

```

Antes: 541 linhas x 45 colunas
Depois: 541 linhas x 42 colunas
Valores ausentes restantes: 0
Colunas de texto restantes: 0

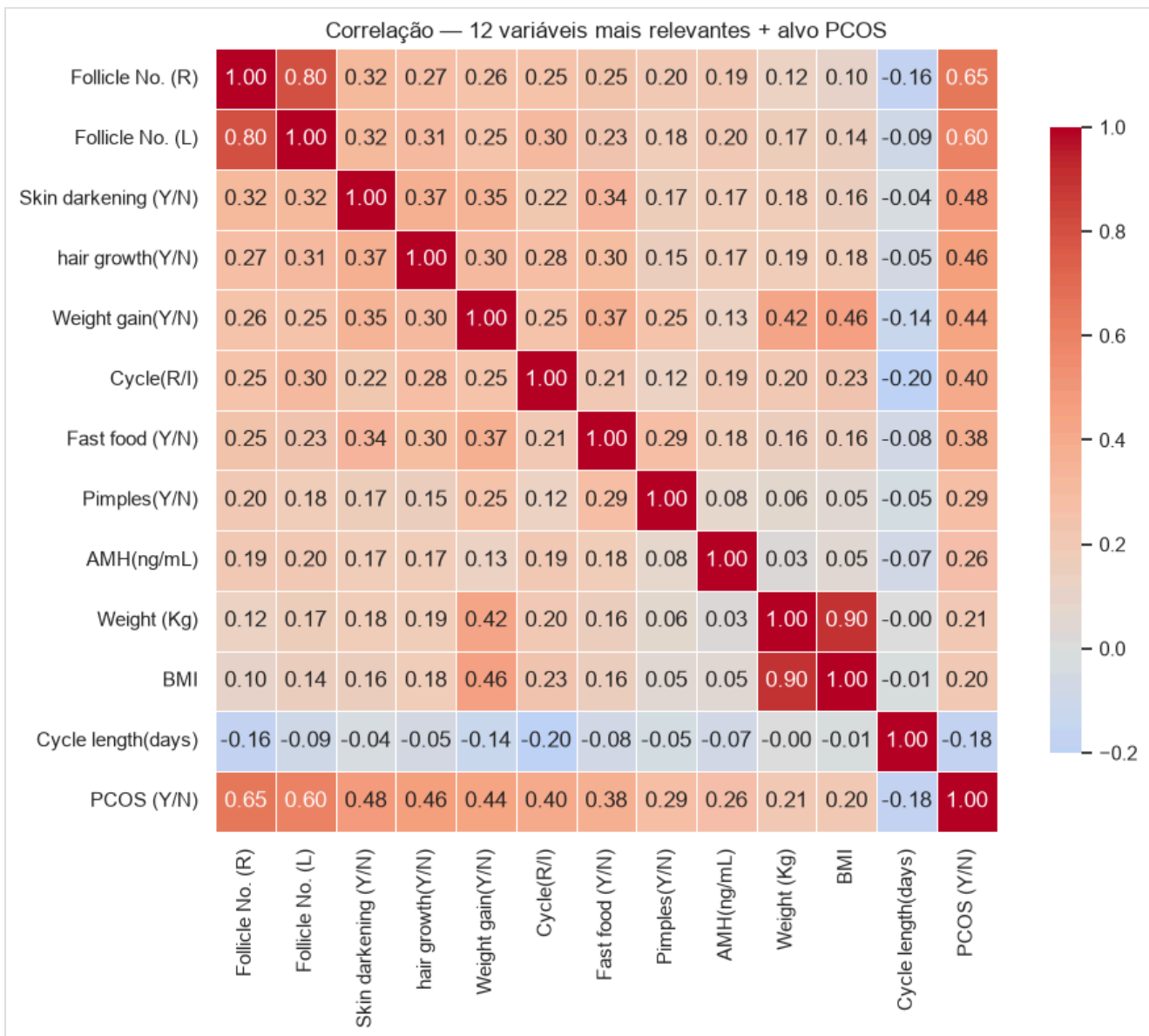
```

3.1 Mapa de correlação (heatmap)

Agora que os dados estão limpos e todos numéricos, pude visualizar as correlações entre as principais variáveis e o alvo em um único gráfico.

```
# Pegamos as 12 variáveis mais correlacionadas com o alvo + o próprio alvo
corr = df.corr(numeric_only=True)
top_feats = corr[TARGET].drop(TARGET).abs().sort_values(ascending=False).head(12).index.tolist()
top_feats = top_feats + [TARGET]

plt.figure(figsize=(10, 8))
sns.heatmap(df[top_feats].corr(), annot=True, fmt=".2f", cmap="coolwarm",
            center=0, square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title("Correlação — 12 variáveis mais relevantes + alvo PCOS")
plt.tight_layout()
plt.show()
```



Leitura: variáveis como o número de folículos (nos dois ovários), o ganho de peso, a irregularidade do ciclo menstrual e o escurecimento da pele aparecem entre as mais correlacionadas com a SOP — exatamente o que se espera do quadro clínico da doença. Isso é um bom sinal de que os dados são coerentes.

4. Modelagem

4.1 Separação treino / teste

Separei os dados em treino (para o modelo aprender) e teste (para avaliar em dados que ele nunca viu). Usei:

- `test_size=0.2` → 20% dos dados para teste;
- `stratify=y` → mantém a mesma proporção de SOP nos dois conjuntos (importante por causa do desbalanceamento).

```
# X = variáveis de entrada (todas menos o alvo); y = alvo
X = df.drop(columns=[TARGET])
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)

print(f"Treino: {X_train.shape[0]} pacientes | Teste: {X_test.shape[0]} pacientes")
print(f"% de SOP no treino: {y_train.mean()*100:.1f}% | no teste: {y_test.mean()*100:.1f}%")
```

```
Treino: 432 pacientes | Teste: 109 pacientes
% de SOP no treino: 32.6% | no teste: 33.0%
```

4.2 Treinando os modelos

Comparei três algoritmos de classificação bem conhecidos:

- Regressão Logística — simples e fácil de interpretar;
- Random Forest — combina muitas árvores de decisão, costuma ter boa performance;
- KNN (K-vizinhos mais próximos) — classifica olhando os pacientes mais parecidos.

Cada modelo é colocado dentro de um `Pipeline` com um `StandardScaler`, que padroniza a escala das variáveis. Usar o scaler dentro do pipeline garante que ele aprenda a escala apenas com os dados de treino, evitando o *data leakage* (vazamento de informação do teste para o treino).

```
def criar_pipeline(modelo):
    """Junta o padronizador (StandardScaler) com o modelo em um único Pipeline."""
    return Pipeline([("scaler", StandardScaler()), ("model", modelo)])

# Dicionário com os 3 modelos que vamos comparar
modelos = {
    "Regressão Logística": LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(random_state=RANDOM_STATE),
    "KNN": KNeighborsClassifier(n_neighbors=5),
}

# Treina e avalia cada modelo, guardando os resultados
resultados = {}
for nome, modelo in modelos.items():
    pipe = criar_pipeline(modelo)
    pipe.fit(X_train, y_train) # treina
    y_pred = pipe.predict(X_test) # prevê no teste
    y_proba = pipe.predict_proba(X_test)[ :, 1] # probabilidade da classe "tem SOP"

    resultados[nome] = {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_proba),
        "pipe": pipe, "pred": y_pred, "proba": y_proba,
    }
    print(f"{nome:22s} -> Accuracy={resultados[nome]['Accuracy']:.3f} | "
          f"Recall={resultados[nome]['Recall']:.3f} | F1={resultados[nome]['F1']:.3f}")
```

```
Regressão Logística    -> Accuracy=0.890 | Recall=0.861 | F1=0.838
Random Forest          -> Accuracy=0.899 | Recall=0.778 | F1=0.836
KNN                    -> Accuracy=0.899 | Recall=0.722 | F1=0.825
```

5. Avaliação e escolha da métrica

Qual métrica priorizar?

Em um problema de diagnóstico médico, nem todos os erros têm o mesmo peso:

- **Falso Negativo (FN):** dizer que a paciente NÃO tem SOP quando ela TEM.

→ É o erro mais grave: um diagnóstico perdido.

- **Falso Positivo (FP):** dizer que tem SOP quando não tem.

→ Menos grave: leva a exames extras, mas não deixa a doença passar.

Por isso, priorizei o **Recall** da classe positiva (proporção de pacientes com SOP que o modelo consegue identificar) e usei o **F1** como equilíbrio entre Recall e Precision.

A **accuracy** sozinha pode enganar quando as classes estão desbalanceadas.

Relembrando as fórmulas:

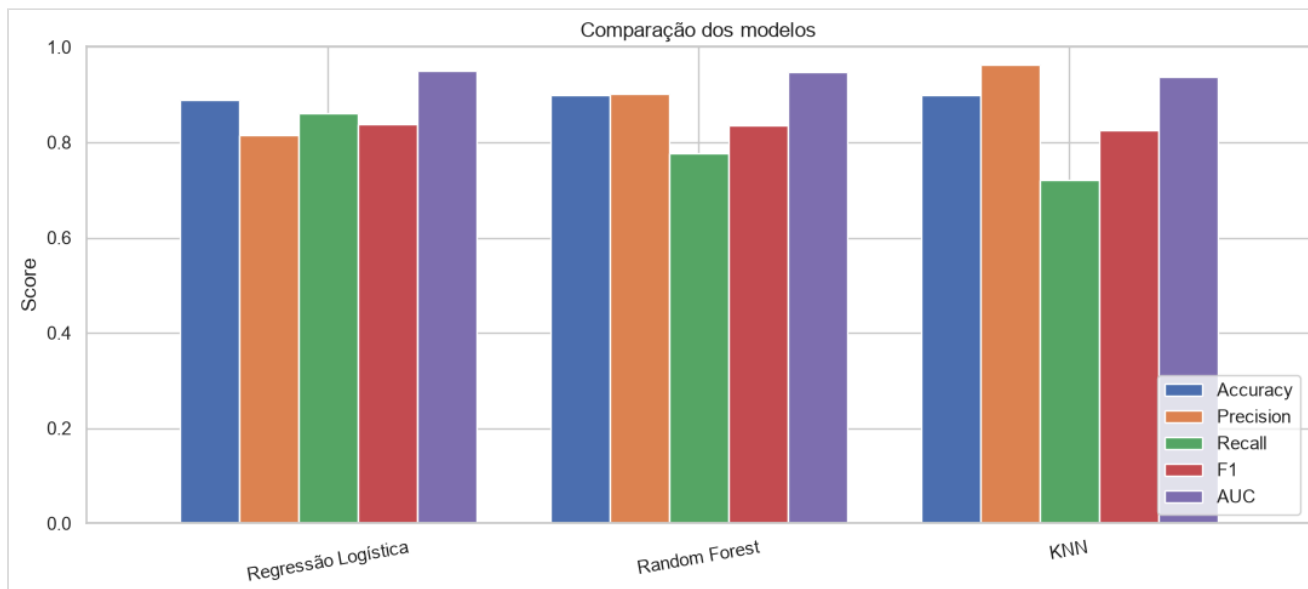
- Accuracy = acertos totais ÷ total de casos
- Precision = TP ÷ (TP + FP) → das que o modelo disse "tem SOP", quantas realmente têm

- $\text{Recall} = \text{TP} \div (\text{TP} + \text{FN}) \rightarrow$ das que realmente têm SOP, quantas o modelo achou
- F1 = média harmônica entre Precision e Recall

```
# Monta uma tabela comparativa e ordena por Recall (e depois F1)
metricas = ["Accuracy", "Precision", "Recall", "F1", "AUC"]
tabela = pd.DataFrame(
    {nome: {m: resultados[nome][m] for m in metricas} for nome in resultados}
).T
tabela = tabela.sort_values(["Recall", "F1"], ascending=False).round(3)
print(tabela.to_string())

# Gráfico comparativo
tabela[metricas].plot(kind="bar", figsize=(11, 5), width=0.8)
plt.title("Comparação dos modelos"); plt.ylabel("Score"); plt.ylim(0, 1)
plt.xticks(rotation=10); plt.legend(loc="lower right")
plt.tight_layout(); plt.show()
```

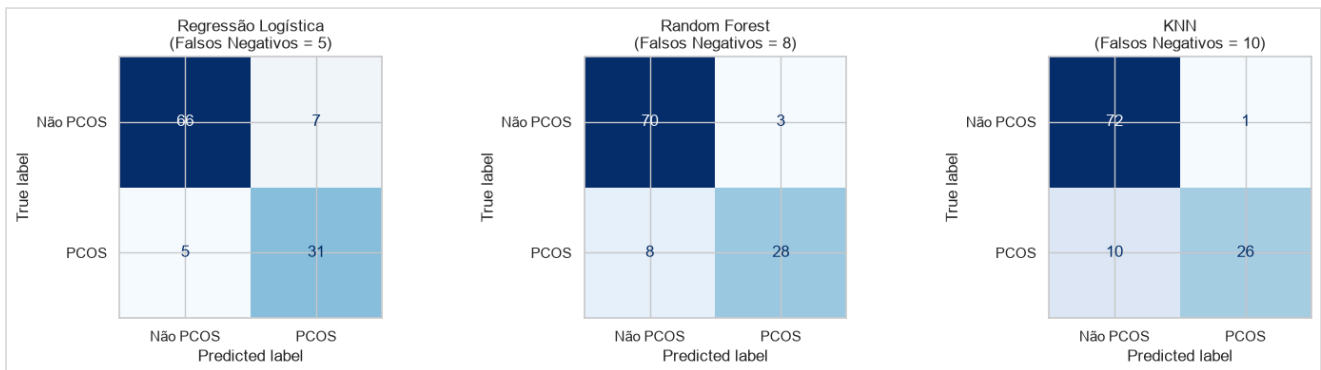
	Accuracy	Precision	Recall	F1	AUC
Regressão Logística	0.890	0.816	0.861	0.838	0.951
Random Forest	0.899	0.903	0.778	0.836	0.948
KNN	0.899	0.963	0.722	0.825	0.937



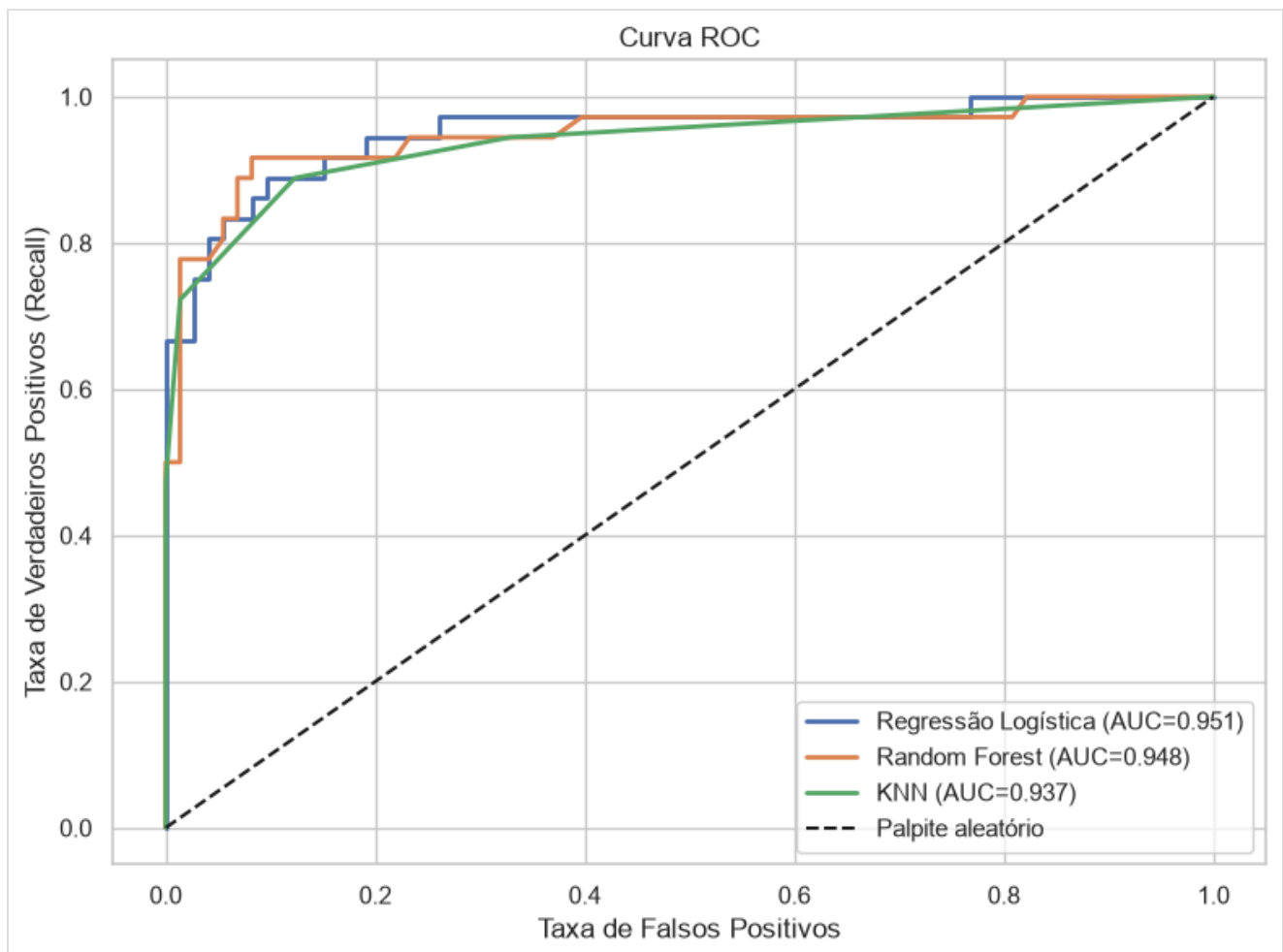
5.1 Matriz de confusão e curva ROC

A matriz de confusão mostra os acertos e erros de cada tipo. O que busquei, principalmente, foi ter, poucos Falsos Negativos (canto inferior esquerdo). A curva ROC resume a capacidade de cada modelo de separar as duas classes — quanto mais próxima do canto superior esquerdo, melhor.


```
# Matrizes de confusão dos 3 modelos, lado a lado
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, nome in zip(axes, resultados):
    cm = confusion_matrix(y_test, resultados[nome]["pred"])
    ConfusionMatrixDisplay(cm, display_labels=["Não PCOS", "PCOS"]).plot(
        ax=ax, cmap="Blues", colorbar=False)
    fn = cm[1, 0] # real = PCOS, previsto = Não -> falso negativo
    ax.set_title(f"{nome}\n(Falsos Negativos = {fn})")
plt.tight_layout(); plt.show()
```



```
# Curva ROC dos 3 modelos
plt.figure(figsize=(8, 6))
for nome in resultados:
    fpr, tpr, _ = roc_curve(y_test, resultados[nome]["proba"])
    plt.plot(fpr, tpr, linewidth=2, label=f'{nome} (AUC={resultados[nome][\"AUC\"]:.3f})')
plt.plot([0, 1], [0, 1], "k--", label="Palpite aleatório")
plt.xlabel("Taxa de Falsos Positivos"); plt.ylabel("Taxa de Verdadeiros Positivos (Recall)")
plt.title("Curva ROC"); plt.legend(loc="lower right")
plt.tight_layout(); plt.show()
```



5.2 Escolha do modelo final

Escolhi como modelo final aquele com o maior Recall (prioridade em não deixar passar casos de SOP). Em caso de empate, o F1 desempata. Abaixo mostro o relatório completo de classificação desse modelo.

```
# Escolhe o melhor modelo pelo Recall (primeiro da tabela já ordenada)
modelo_final_nome = tabela.index[0]
modelo_final = resultados[modelo_final_nome]["pipe"]
y_pred_final = resultados[modelo_final_nome]["pred"]

print(f"Modelo final escolhido: {modelo_final_nome}\n")
print(classification_report(y_test, y_pred_final,
                           target_names=["Não PCOS (0)", "PCOS (1)"]))
```

Modelo final escolhido: Regressão Logística

	precision	recall	f1-score	support
Não PCOS (0)	0.93	0.90	0.92	73
PCOS (1)	0.82	0.86	0.84	36
accuracy			0.89	109
macro avg	0.87	0.88	0.88	109
weighted avg	0.89	0.89	0.89	109

6. Explicabilidade

Em saúde, não basta o modelo acertar — é preciso entender por quê ele decide de certa forma. Isso torna o modelo auditável e ajuda o médico a confiar (ou desconfiar) da sugestão. Utilizei duas técnicas:

- Feature Importance — visão global de quais variáveis o modelo mais usa;
- SHAP — explica a contribuição de cada variável, inclusive por paciente.

6.1 Feature Importance

Se o modelo final for uma Random Forest, utilizei a importância das variáveis calculada pela própria árvore. Se for um modelo linear (Regressão Logística), utilizei o valor absoluto dos coeficientes como medida de importância.

```

modelo_step = modelo_final.named_steps["model"]

if hasattr(modelo_step, "feature_importances_"):
    # Modelos de árvore (Random Forest) têm feature_importances_
    importancia = pd.Series(modelo_step.feature_importances_, index=X.columns)
    titulo = "Feature Importance (Random Forest)"
else:
    # Modelos lineares: usamos o |coeficiente| como importância
    importancia = pd.Series(np.abs(modelo_step.coef_[0]), index=X.columns)
    titulo = "Importância por |coeficiente| (Regressão Logística)"

top15 = importancia.sort_values(ascending=False).head(15)

plt.figure(figsize=(9, 7))
sns.barplot(x=top15.values, y=top15.index, hue=top15.index,
            palette="viridis", legend=False)
plt.title(f"{titulo} - {modelo_final_nome}")
plt.xlabel("Importância"); plt.tight_layout(); plt.show()

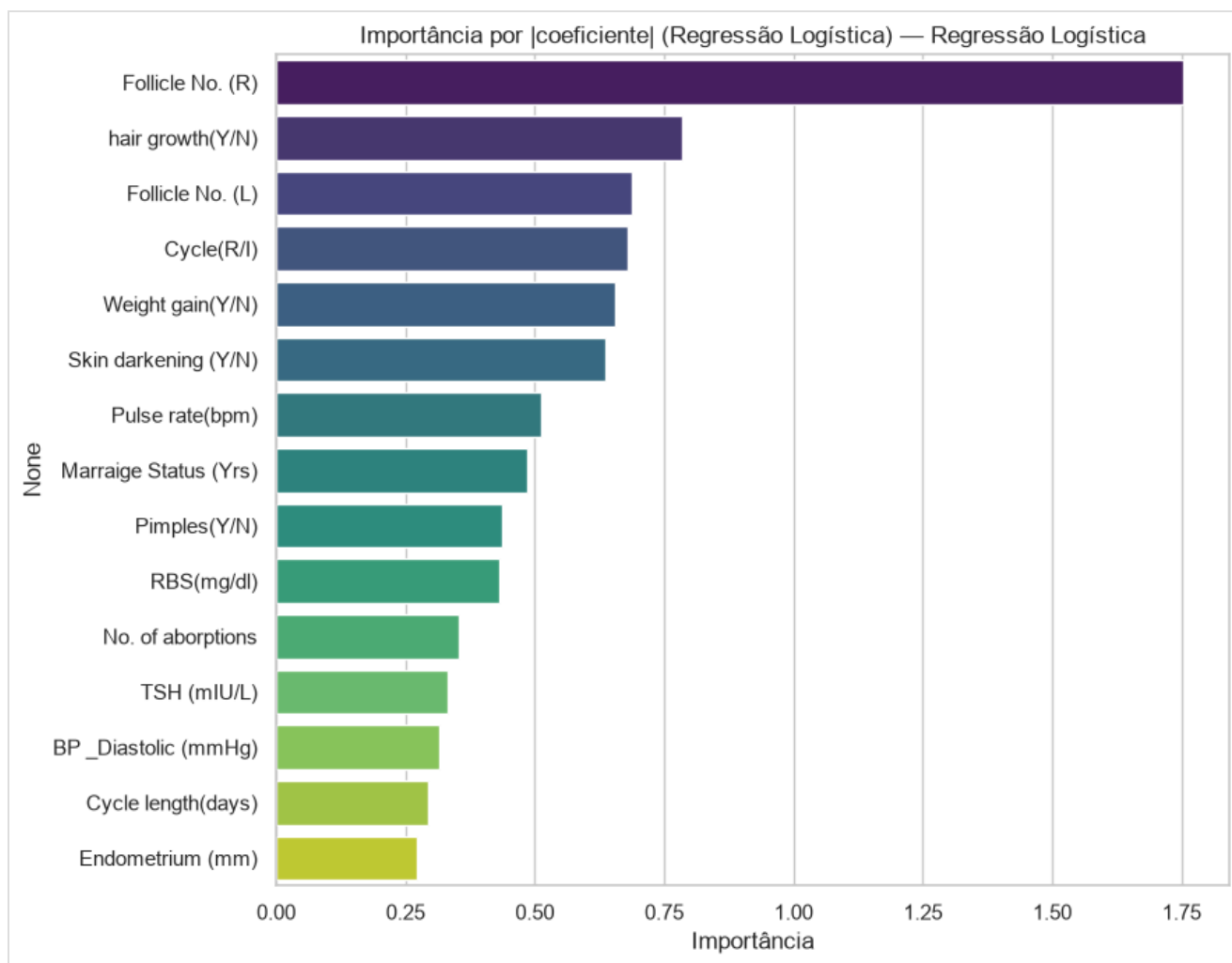
print("Top 5 variáveis mais influentes:")
print(top15.head(5).round(3))

```

```

Top 5 variáveis mais influentes:
Follicle No. (R)    1.755
hair growth(Y/N)   0.785
Follicle No. (L)    0.686
Cycle(R/I)          0.678
Weight gain(Y/N)    0.656
dtype: float64

```



Leitura: se variáveis como número de folículos, AMH (hormônio anti-mülleriano), ganho de peso e irregularidade do ciclo aparecem no topo, é um ótimo sinal — são justamente os fatores associados à SOP na prática clínica. O modelo aprendeu padrões coerentes, e não "atalhos" sem sentido.

6.2 SHAP

O **SHAP** (*SHapley Additive exPlanations*) atribui a cada variável um valor que indica o quanto ela empurrou a previsão para "tem SOP" ou "não tem SOP", em cada paciente. O gráfico de resumo (*summary plot*) junta essa informação de todas as pacientes do teste.

```

import shap

modelo_step = modelo_final.named_steps["model"]

# Escolhemos o explainer conforme o tipo do modelo final
if hasattr(modelo_step, "feature_importances_"):
    # Modelo de árvore -> TreeExplainer (usa os dados originais)
    explainer = shap.TreeExplainer(modelo_step)
    X_exp = X_test
else:
    # Modelo linear -> LinearExplainer (usa os dados já escalonados pelo pipeline)
    scaler = modelo_final.named_steps["scaler"]
    X_exp = pd.DataFrame(scaler.transform(X_test), columns=X.columns, index=X_test.index)
    explainer = shap.LinearExplainer(modelo_step, X_exp)

shap_values = explainer.shap_values(X_exp)

# Garante que estamos olhando a classe positiva (PCOS = 1),
# compatível com diferentes versões do SHAP.
if isinstance(shap_values, list):
    shap_pcos = shap_values[1]
elif hasattr(shap_values, "ndim") and shap_values.ndim == 3:
    shap_pcos = shap_values[:, :, 1]
else:
    shap_pcos = shap_values

shap.summary_plot(shap_pcos, X_exp, show=False)
plt.title("SHAP – impacto das variáveis na previsão de PCOS")
plt.tight_layout(); plt.show()

```

Background dataset has 109 samples but max_samples=100. Subsampling to 100 samples for SHAP value computation. To use all samples, set max_samples=109 when initializing the masker.



Como ler o gráfico SHAP:

- Cada ponto é uma paciente. Pontos à direita empurram a previsão para "tem SOP";

pontos à esquerda, para "não tem".

- A cor indica o valor da variável: vermelho = valor alto, azul = valor baixo.
- Exemplo típico: muitos folículos (vermelho) aparecem à direita → aumentam a probabilidade

de SOP. Isso confirma, de forma visual e por paciente, o que vimos na feature importance.

7. Discussão crítica

O modelo pode ser usado na prática? Como?

Sim — mas como ferramenta de apoio / triagem, e não como decisor final. Ele pode priorizar pacientes com maior probabilidade de SOP para uma investigação mais rápida, ajudando a acelerar o atendimento em locais com grande volume de pacientes.

Por que priorizamos o Recall?

Porque o erro mais grave neste contexto é o falso negativo — deixar de identificar uma paciente que realmente tem SOP. Um recall alto reduz esse risco. O F1 nos garante que isso não vem ao custo de "chutar SOP para todo mundo".

Por que a explicabilidade importa?

As técnicas de Feature Importance e SHAP tornam a decisão transparente e auditável.

O médico consegue ver quais fatores levaram à sugestão do modelo e comparar com seu próprio julgamento clínico, o que aumenta a confiança na ferramenta.

Limite ético — o médico tem a palavra final.

Este é um sistema de apoio. O diagnóstico definitivo é sempre responsabilidade do profissional de saúde. O modelo não substitui exames, avaliação clínica nem o julgamento médico.

Limitações e próximos passos.

- O dataset é relativamente pequeno (~541 pacientes) e vem de uma origem específica, o que limita a generalização.
- Antes de qualquer uso real, seria necessária validação externa com dados de outras populações e hospitais.
- Possíveis melhorias: coletar mais dados, ajustar hiperparâmetros, testar o ajuste do limiar de decisão para aumentar ainda mais o Recall, e monitorar o modelo ao longo do tempo.